Streams and λ expression reuse via Predicate and Function interfaces

Student Worksheet

Block 1: map(), flatMap(), reduce() applied to graphs

In this block, you will work with an undirected graph represented as a list of lists. Each position i contains the list of nodes adjacent to node i. The provided Java skeleton defines a small example graph with nodes 0, 1, 2, 3, 4 and one method per exercise that you must complete.

In all exercises you must:

- Use only basic Java and the functional operations seen in class.
- Build Streams from collections like `List<Integer>`.
- Use operations `map`, `flatMap`, `distinct`, `sorted`, and `reduce` when indicated.
- Show results in console with `System.out.println`.
- Do not modify the method signatures in the skeleton.

Below are the descriptions of the exercises you must solve by completing the code marked as `// TODO` in the project file.

1 Exercise 1: Node degrees with map

Required: Complete the `exercise1_NodeDegrees` method to:

1. Transform each adjacency list into its size using `map`.
2. Build a list with the degrees of all nodes.
3. Print the full list of degrees to the console.
4. Also print, node by node, its corresponding degree.

The goal is to practice basic usage of `map` on the graph structure.

2 Exercise 2: Neighbors of neighbors with flatMap

Required: Complete the `exercise2_NeighborsOfNeighbors` method for a starting node. You must:

1. Take the list of direct neighbors of the starting node.
2. Apply `flatMap` to extract, for each neighbor v, all its neighbors.

3. Obtain a list that may contain duplicates.
4. Display on console:
   - The direct neighbors of the starting node.
   - The full list of neighbors of neighbors (without removing duplicates).

The goal is to observe how `flatMap` decomposes a structure of structures.

3 Exercise 3: Second-level neighbors without duplicates

Required: Complete the `exercise3_SecondLevelNeighborsUnique` method to:

1. Obtain neighbors of neighbors using `flatMap`.
2. Remove duplicates using `distinct()`.
3. Exclude:
   - The starting node.
   - Its direct neighbors.
4. Sort the result with `sorted()`.
5. Display the resulting list.

This exercise combines several functional transformations into a single Stream.

4 Exercise 4: Counting edges with map and reduce

Required: Complete the `exercise4_CountEdges` method to:

1. Transform each adjacency list into its size using `map`.
2. Calculate the total sum of degrees using `reduce`.
3. Take into account that, since it is an undirected graph, each edge appears twice.
4. Calculate the number of edges as half the sum of degrees.
5. Display both the sum of degrees and the total number of edges in the console.

The goal is to see how `map` and `reduce` allow operating on the graph structure in a declarative way.

5 Exercise 5: Nodes with maximum degree

Required: Complete the `exercise5_MaxDegreeNodes` method:

1. Calculate the degree of each node.
2. Get the maximum degree using `max()` or an equivalent reduction.
3. Find all nodes that have that maximum degree.
4. Return or display the sorted list of nodes with maximum degree.

This exercise combines mapping, filtering, and searching for values that meet specific conditions.

Block 2: Lambda function reuse

In this block you will practice how to reuse functional code in Java. In particular, you will learn to build functions, predicates, and consumers that can be combined and composed with each other to avoid duplicating logic.

In the provided Java skeleton you will find five empty methods. Each corresponds to an exercise you must complete using only:

- Basic functional interfaces: `Predicate<T>`, `Function<T,R>`, `Consumer<T>`.
- Composition using `and()`, `or()`, `negate()`, `andThen()`.
- Immutable collections with `List.of(...)`.
- Printed messages with `System.out.println`.

The goal is to learn to create functional pieces that can be cleanly reused in various contexts.

6 Exercise 1: Filtering with reusable predicates

Required: Complete the `exercise1_FilterReusablePredicates()` method:

1. Create an immutable list of integers with `List.of(...)`.
2. Define two reusable predicates:
   - `isEven`: checks if a number is even.
   - `isPositive`: checks if a number is greater than 0.
3. Build the combined predicate `isEven.and(isPositive)`.
4. Filter the list using a Stream and display the result on the console.

The goal is to see how simple predicates can be composed to build more complex criteria without duplicating code.

7 Exercise 2: Reusable predicates for a simple graph

Required: Complete the `exercise2_GraphPredicates()` method:

1. Represent an undirected graph using an adjacency list (each position contains the list of neighbors of a node).
2. Define reusable predicates, for example:
   - `isNeighborOf(k)`: returns a predicate that checks if a node is a neighbor of k. *Note:* Reflect if it is compatible with `filterNodes` and the possible need for a `BiPredicate`.
   - `isLeafNode`: checks if a node has exactly one neighbor.
3. Use these predicates to filter nodes and display the results.

The purpose is to learn to generate parameterized predicates that can be used in multiple calls.

8 Exercise 3: Function reuse with Function<T,R>

Required: Complete the `exercise3_FunctionReusable()` method:

1. Define two reusable functions on integers:
   - `doubleValue`: multiplies by 2.
   - `increment`: adds 1.
2. Build a composition: `doubleValue.andThen(increment)`.
3. Apply it to several values and display the results.

This exercise reinforces how to combine functions without rewriting code.

9 Exercise 4: Reusable Consumers (order of effects)

Required:

1. Define several `Consumer<Integer>` on nodes. For example:
   - `print`: simply prints the node, e.g., with a message `"Node :  " + <the node>`.
   - `debug`: prints with a more detailed debug message like `"[debug] visiting " + x`.
   - `printSquared`: prints the square of the node value.
2. Use `visitNodes` with individual Consumers.
3. Compose Consumers with `andThen` to observe how the order of effects changes the output.

10 Exercise 5: Reusing predicates to validate nodes

Required:

1. Define a simple graph with node indices 0..4.
2. Define reusable predicates to check if a node index is valid (e.g., greater than or equal to zero and less than the graph size).
3. Build a composite predicate `validIndex`.
4. Ensure your code is compatible with this fragment:

```
Predicate<Integer> validIndex = positiveIndex.and(lessThanSize);
```

5. Validate different paths (lists of nodes) using a helper method that receives the predicate.

11 Optional Exercise 5: Reusable validation of paths in a graph

Required: Complete the `exercise5_PathValidationReusable()` method:

1. Use a graph represented as a list of adjacency lists as explained initially.
2. Define a predicate `areAdjacent(i, j)` that checks if an edge exists between nodes i and j. *Note:* Keep in mind that a two-argument predicate requires the `BiPredicate` functional interface. Specifically for this case consider something like:

```
BiPredicate<Integer, Integer> areAdjacent ...
```

3. Build a path validator that checks that: each consecutive pair of nodes in a list are adjacent.
4. Apply the validator to several routes and show if each one is valid or not.

This exercise allows practicing parameterized predicates, composition, and functional validation.